b BASE + group + b×4

n BASE + n×0x20

FENCE.I Zifencei

Zicbom


```
                    +-----------------------------------------------------+
                    |           RV32IMAC SoC Top Level                    |
                    |                                                     |
  +-----------------+-----------------------------------------------+     |
  |  KV32 Core      |                                               |     |
  |  +------------+ |                                               |     |
  |  |   IF       |-+->Instruction                                  |     |
  |  |   (Fetch)  |<+--  Buffer                                     |     |
  |  +-----+------+ |   (IB_DEPTH)                                    |     |
  |        |        |        |                                        |     |
  |  +-----v------+ |        v                                        |     |
  |  |   ID       | |   +----------+         +-------------+          |     |
  |  |  (Decode)  | |   | I-Cache  |<------->|  Mem Arbiter|          |     |
  |  +-----+------+ |   | (4KB/2W) |         |  (I+D paths)|          |     |
  |        |        |   +----------+         +------+------+          |     |
  |  +-----v------+ |                               |                 |     |
  |  |   EX       | |                               v                 |     |
  |  | (Execute)  | |                       +-------------+          |     |
  |  +-----+------+ |                       | AXI4-Lite   |          |     |
  |        |        |                       |  Crossbar   |          |     |
  |  +-----v------+ |   +----------+         |  (1-to-11)  |          |     |
  |  |   MEM      |-+-->| D-Cache  |<------->+------+------+          |     |
  |  |  (Memory)  |<+---| (4KB/2W) |                |                 |     |
  |  +-----+------+ |   |write-back|                |                 |     |
  |        | Store  |   +----------+                |                 |     |
  |  +-----v------+ |   Buffer(SB)                  |                 |     |
  |  |   WB       | |   +---------------------------+-----------------+---+
  |  | (Writeback)| |   | Slave 0: RAM (2MB)        |                 |     |
  |  +------------+ |   | 0x8000_0000-0x801F_FFFF   v                 |     |
  |                 |   |                     +-------------+          |     |
  |                 |   |                     |  AXI4 RAM   |          |     |
  +-----------------+   |                     |  (2MB ext)  |          |     |
                        |                     +-------------+          |     |
                        |                                              |     |
                        | Slave 1: Magic    0x4000_0000-0x4000_FFFF    |     |
                        |   +--------------------------+               |     |
                        |   | Simulation console I/O   |               |     |
                        |   | Exit code control        |               |     |
                        |   +--------------------------+               |     |
                        |                                              |     |
                        | Slave 2: CLINT    0x0200_0000-0x020B_FFFF    |     |
                        |   +--------------------------+               |     |
                        |   | Timer (mtime/mtimecmp)   |               |     |
                        |   | Software IRQ (msip)      |---->timer_irq|     |
                        |   +--------------------------+   +>sw_irq    |     |
```

```
|                                                |   |
| Slave 3: PLIC     0x0C00_0000-0x0CFF_FFFF      |   |
|   +--------------------------+                 |   |
|   | Priority/Enable/Claim    |<---uart_irq     |   |
|   | Threshold/Complete       |<---spi_irq      |   |
|   +--------------+-----------+<---i2c_irq      |   |
|                  |            <---dma_irq      |   |
|                  +-------------->ext_irq       |   |
|                                                |   |
| Slave 4: DMA      0x2000_0000-0x2000_FFFF      |   |
|   +--------------------------+                 |   |
|   | 4 channels, 1D/2D/3D/SG  |<-->(AXI4        |   |
|   | 32-bit data width        |    master       |   |
|   | Performance counters     |    port)        |   |
|   +--------------------------+                 |   |
|                                                |   |
| Slave 5: UART     0x2001_0000-0x2001_FFFF      |   |
|   +--------------------------+                 |   |
|   | TX/RX FIFOs (16-deep)    |<---uart_rx      |   |
|   | Baud rate: 25MHz max     |--->uart_tx      |   |
|   +--------------------------+                 |   |
|                                                |   |
| Slave 6: I2C      0x2002_0000-0x2002_FFFF      |   |
|   +--------------------------+                 |   |
|   | Master mode (100k/400k)  |<-->i2c_sda      |   |
|   | TX/RX FIFOs (8-deep)     |<-->i2c_scl      |   |
|   +--------------------------+                 |   |
|                                                |   |
| Slave 7: SPI      0x2003_0000-0x2003_FFFF      |   |
|   +--------------------------+                 |   |
|   | Master mode (4 CS lines) |--->spi_sclk     |   |
|   | TX/RX FIFOs (8-deep)     |--->spi_mosi     |   |
|   | Modes 0-3 (CPOL/CPHA)    |<---spi_miso     |   |
|   +--------------------------+--->spi_cs_n     |   |
|                                                |   |
| Slave 8: Timer    0x2004_0000-0x2004_FFFF      |   |
|   +--------------------------+                 |   |
|   | 4 ch, 32-bit up-counter  +-->pwm[3:0]      |   |
|   | PWM + dual compare       +-->timer_irq     |   |
|   +--------------------------+                 |   |
|                                                |   |
| Slave 9: GPIO     0x2005_0000-0x2005_FFFF      |   |
|   +--------------------------+                 |   |
|   | 4 banks x 32 pins        |<-->gpio[127:0]|  |
|   | Dir/IRQ/edge/level       +-->gpio_irq      |   |
|   +--------------------------+                 |   |
|                                                |   |
| Slave 10: WDT     0x2006_0000-0x2006_FFFF      |   |
|   +--------------------------+                 |   |
|   | 32-bit countdown timer   +-->wdt_reset_o|  |   |
|   | IRQ or hw-reset on expiry+-->wdt_irq       |   |
```

```
                          |   +-------------------------+              |   |
                          +----------------------------------------------+   |
                                                                             |
    External Pins:                                                           |
      clk, rst_n                                                             |
      uart_tx, uart_rx                                                       |
      spi_sclk, spi_mosi, spi_miso, spi_cs_n[3:0]                            |
      i2c_scl_o, i2c_scl_i, i2c_scl_oe                                       |
      i2c_sda_o, i2c_sda_i, i2c_sda_oe                                       |
      gpio_out[127:0], gpio_in[127:0], gpio_oe[127:0]                        |
      pwm_out[3:0]                                                           |
      m_axi_* (external RAM interface)                                       |
                                                                             |
```

kv32_icache

pmacfg0 pmacfg1 pmaaddr0 pmaaddr7

addr[31]=1 " addr[31]=0 "

axi_uart.sv

axi_i2c.sv

axi_spi.sv

axi_dma.sv

axi_gpio.sv !

axi_timer.sv

kv32_pm

WFI

WFI kv32_core core_sleep_o

kv32_pm

```
clk_en = !sleep_req_i       // core not in WFI
       | timer_irq_i         // CLINT machine-timer
       | external_irq_i      // PLIC external IRQ
       | software_irq_i      // CLINT software IRQ
       | irq_wake_pending;   // captured single-cycle pulse
```

clk_i

clk_en=1 irq_wake_pending sleep_req_i

Table 1. Clock-Gate Implementation

	BUFGCE CE_TYPE="SYNC"
	clk_i

core_sleep_o

Table 2. Complete Memory Map

			!

axi_xbar

addr[31:21] == 11'h400

addr[31:16] == 16'h4000

addr[31:20] == 12'h020

addr[31:24] == 8'h0C

addr[31:16] == 16'h2000

```
   addr[31:16] == 16'h2001
addr[31:16] == 16'h2002
addr[31:16] == 16'h2003
   addr[31:16] == 16'h2004
  addr[31:16] == 16'h2005
   addr[31:16] == 16'h2006
```

Table 3. Supported ISA Extensions

	`kv32_rvc`

Table 4. Pipeline Stages

" " "

Table 5. Performance Metrics

Table 6. Implemented CSRs

			[7]=L [4:3]=A [2]=X [1]=C [0]=B

Table 7. Exception Causes (mcause[31]=0)

Table 8. Interrupt Causes (mcause[31]=1)

`mepc`

`mcause`

`mtval`

`mstatus.MIE`

`mtvec`

`MRET` `mepc`

Table 9. I-Cache Configuration

	!

```
#
#
#
#

#                    !
#                                    !
#                                      !
```

`imem_req_valid`

`imem_resp_data`

`cmo_*`

Table 10. Supported CMO Operations

`FENCE.I` `cbo.inval`

`perf_req_cnt`

`perf_hit_cnt`

`perf_miss_cnt`

`perf_bypass_cnt`

`perf_fill_cnt`

`perf_cmo_cnt`

`kv32_dcache`

Table 11. D-Cache Configuration

	!

```
#
#
#
#

#                    !
#                    !
#                                  !
#                                    !
```

`core_req_valid` `mem_read=1`

`S_IDLE`

core_resp_data

" ! "

!

S_HIT_WR

S_WT_AW → S_WT_W → S_WT_B

DCACHE_WRITE_ALLOC=1

DCACHE_WRITE_ALLOC=0

use_dcache=0

`cmo_*`

Table 12. Supported CMO Operations

`CMO_FLUSH_ALL` `FENCE.I`

`perf_req_cnt`

`perf_hit_cnt`

`perf_miss_cnt`

`perf_bypass_cnt`

`perf_fill_cnt`

`perf_evict_cnt`

`perf_cmo_cnt`

Table 13. UART Features

Table 14. UART Registers

			"

`(IE & IS) != 0`

`BAUD_RATE`

```
CLKS_PER_BIT = CLK_FREQ / BAUD_RATE
```

`CLKS_PER_BIT >= 4`

`CLKS_PER_BIT >= 8`

Table 15. Common Baud Rate Examples (100 MHz Clock)

`CLKS_PER_BIT < 4`

Table 16. UART Pins

Table 17. SPI Features

	`SCLK = CLK / (2 * (DIV + 1))`

Table 18. SPI Registers

			"
			`SCLK = CLK / (2 * (DIV + 1))` " "

Table 19. SPI Clock Modes

Table 20. Common SPI Clock Frequencies

Table 21. SPI Pins

Table 22. I2C Features

Table 23. I2C Registers

			`SCL_period = CLK / (4 * (DIV + 1))`

Table 24. Common I2C Speeds

#

#

#

Table 25. I2C Pins

$

Table 26. DMA Features

`BASE + channel * 0x40`

*Table 27. Per-Channel Registers (offset from BASE + ch*0x40)*

Table 28. Global Registers (offset from BASE)

SRC_ADDR DST_ADDR XFER_CNT

CTRL[4:3]=00

CTRL[5:6]

CTRL[1]=1

SRC_ADDR DST_ADDR XFER_CNT

SRC_STRIDE DST_STRIDE

ROW_CNT

CTRL[4:3]=01

! !

```
XFER_CNT = 16 * 4        // 16 pixels x 4 bytes/pixel
SRC_STRIDE = 640 * 4     // Source pitch
```

```
DST_STRIDE = 16 * 4      // Destination pitch (packed)
ROW_CNT = 16             // 16 rows
```

SRC_PLANE_STRIDE DST_PLANE_STRIDE PLANE_CNT

CTRL[4:3]=10

{src_addr, dst_addr, xfer_cnt, mode_ctrl}

SG_ADDR

SG_CNT

CTRL[4:3]=11

IRQ_STAT

IRQ_STAT

```
DMA_PERF_CTRL = 0x3;    // Enable counters and reset
// ... run workload ...
cycles = DMA_PERF_CYCLES;
rd_bytes = DMA_PERF_RD_BYTES;
wr_bytes = DMA_PERF_WR_BYTES;
bandwidth_MBps = (rd_bytes + wr_bytes) / cycles * CLK_FREQ / 1e6;
```

MAX_BURST_LEN

Table 29. GPIO Features

	`{VERSION[31:16], NUM_BANKS[15:8], NUM_PINS[7:0]}` `0x0001_0120`

		b	`BASE + group_offset + b * 4`

Table 30. GPIO Register Map

			`{VERSION[31:16], NUM_BANKS[15:8], NUM_PINS[7:0]}`

`gpio_irq`

Table 31. GPIO External Pins

Table 32. Timer Features

	`{VERSION[31:16], NUM_CHANNELS[15:8], COUNTER_WIDTH[7:0]}` `0x0001_0420`

`BASE + channel * 0x20`

Table 33. Timer Per-Channel Registers (BASE + ch × 0x20)

Table 34. Timer Global Registers

			`{VERSION[31:16], NUM_CHANNELS[15:8], COUNTER_WIDTH[7:0]}`

```
PRESCALE = 0         // No prescaling; clock = 100 MHz
COMPARE2 = 99999     // Period = 100000 ticks = 1 ms
INT_ENABLE = 0x1     // Enable channel 0 global IRQ
CTRL = EN | INT_EN   // Start timer with interrupt
```

```
COMPARE1 = 49999     // Falling edge at 50% duty
COMPARE2 = 99999     // Period = 1 ms
CTRL = EN | PWM_EN | PWM_POL | INT_EN
```

`KV_PLIC_SRC_TIMER0 + N`

Table 35. Timer External Pins

Table 36. WDT Features

	KV_PLIC_SRC_WDT
	wdt_reset_o rst_n

Table 37. WDT Register Map

			wdt_reset_o
			0x0001_0020

0x03 0x01

#

wdt_reset_o *latches* rst_n wdt_reset_o

wdt_irq STATUS[0] & CTRL[1]

wdt_reset_o rst_n

kv32sim exit(2) sim_→htif_exit(2)

Table 38. WDT External Pins

Table 39. CLINT Features

Table 40. CLINT Registers

mtime

```
mtime >= mtimecmp
```

mtimecmp

```
        mtime = read_mtime();
        mtimecmp = mtime +       ;  // 1ms = 100k cycles @ 100MHz
write_mtimecmp(mtimecmp);
```

MSIP[0] = 1 MSIP[0] = 0

Table 41. PLIC Features

	PLIC_SIZE = 0x01000000

Table 42. PLIC Registers

!			

Table 43. PLIC Interrupt Source IDs

		`KV_PLIC_SRC_TIMER0`
		`KV_PLIC_SRC_TIMER1`
		`KV_PLIC_SRC_TIMER2`
		`KV_PLIC_SRC_TIMER3`
		`KV_PLIC_SRC_WDT`

irq_src[i] PENDING[i]=1

priority[i] > threshold

external_irq

CLAIM PENDING[i]

COMPLETE

irq_src[i] PENDING[i]

PENDING[i]

```
PRIORITY[1] = 5;  // UART high priority
PRIORITY[2] = 3;  // SPI medium priority
PRIORITY[3] = 3;  // I2C medium priority
PRIORITY[4] = 1;  // DMA low priority
PRIORITY[5] = 4;  // GPIO medium-high priority
PRIORITY[6] = 2;  // Timer low-medium priority
THRESHOLD = 0;    // Allow all enabled interrupts
ENABLE = 0x7E;    // Enable sources 1-6
```

Table 44. Magic Device Features

Table 45. Magic Device Registers

axi_magic OKAY

0x4000_0000 CONSOLE

0x4000_0004 EXIT

0x4000_1000-0x4000_11FF

SLVERR

addr[8:2]

axi_wdata

```
Pass:     write 1
```

```
Fail:     write ((exit_code << 1) | 1)
```

```
                  *magic_exit = (         *)          ;
                  *magic_console = (         *)          ;

// Print to console
*magic_console = 'H';
*magic_console = 'i';
*magic_console = '\n';

// Exit with success
*magic_exit =  ;  // Simulation terminates with exit(0)
```

Table 46. AXI Protocol Summary

`axi_xbar`

```
if (addr[31:16] == 16'hFFFF)       → Slave 9 (Magic)
else if (addr[31:16] == 16'h2005)  → Slave 8 (Timer)
else if (addr[31:16] == 16'h2004)  → Slave 7 (GPIO)
else if (addr[31:12] == 20'h2003_0) → Slave 6 (DMA)
else if (addr[31:16] == 16'h2002)  → Slave 5 (SPI)
else if (addr[31:16] == 16'h2001)  → Slave 4 (I2C)
else if (addr[31:16] == 16'h2000)  → Slave 3 (UART)
else if (addr[31:26] == 6'h03)     → Slave 2 (PLIC)
else if (addr[31:20] == 12'h020)   → Slave 1 (CLINT)
else if (addr[31:21] == 11'h400)   → Slave 0 (RAM)
else                               → SLVERR (unmapped)
```

SLVERR

OKAY

Table 47. AXI4 Memory Features

ARBURST = 2'b10

ARLEN = (line_size / 4) - 1

ARBURST = 2'b10 ARLEN = (line_size / 4) - 1

AWBURST = 2'b01 AWLEN = WORDS_PER_LINE - 1

AWBURST = 2'b01 AWLEN = 0

ARBURST = 2'b01 ARLEN = 0

```
ARCHR = 0x80000054
```

```
ARLEN = 7           // 8 beats
ARBURST = WRAP
Beats returned:
  0: 0x80000054 (critical word, returned first)
  1: 0x80000058
  2: 0x8000005C
  3: 0x80000040 (wrap to line boundary)
  4: 0x80000044
  5: 0x80000048
  6: 0x8000004C
  7: 0x80000050
```

```
ARADDR = 0x80000014
ARLEN = 15          // 16 beats
ARBURST = WRAP
Beats returned:
  0: 0x80000014 (critical word, returned first)
  1: 0x80000018
  2: 0x8000001C
  3: 0x80000020
  4: 0x80000024
  5: 0x80000028
  6: 0x8000002C
  7: 0x80000030
  8: 0x80000034
  9: 0x80000038
  A: 0x8000003C
  B: 0x80000000 (wrap to line boundary)
  C: 0x80000004
  D: 0x80000008
  E: 0x8000000C
  F: 0x80000010
```

Table 48. AXI Transaction Latencies

Table 49. Clock and Reset Specifications

Table 50. Timing Characteristics (100 MHz, Tclk=10ns)

Table 51. Power Consumption Estimates

Table 52. Operating Conditions (FPGA implementation)

0x8000_0000

elfloader.cpp

```
                (    ) {
// Set trap vector (must be 4-byte aligned)
write_csr(mtvec, (        )trap_entry);

// Enable M-mode interrupts in mstatus
set_csr(mstatus, MSTATUS_MIE);

// Enable specific interrupt types in mie
set_csr(mie, MIE_MEIE | MIE_MTIE | MIE_MSIE);

// Configure PLIC
kv_plic_set_priority(KV_PLIC_SRC_UART,  );
kv_plic_enable_interrupt(KV_PLIC_SRC_UART);
kv_plic_set_threshold( );
```

```
    // Configure CLINT timer for periodic interrupt (1ms)
            mtime = kv_clint_read_mtime();
    kv_clint_write_mtimecmp(mtime +       );  // +1ms @ 100MHz
}
```

```
                (    ) {
            mcause = read_csr(mcause);
            mepc = read_csr(mepc);

       (mcause &           ) {
        // Interrupt
                irq_code = mcause &     ;
              (irq_code) {
             :  // Timer interrupt
            handle_timer_interrupt();
                ;
              : // External interrupt (PLIC)
            handle_external_interrupt();
                ;
             :  // Software interrupt
            handle_software_interrupt();
                ;
        }
    }      {
        // Exception
        handle_exception(mcause, mepc);
    }
}

                        (    ) {
    // Advance mtimecmp for next interrupt
            mtimecmp = kv_clint_read_mtimecmp();
    kv_clint_write_mtimecmp(mtimecmp +       );  // +1ms
}

                            (    ) {
    // Claim interrupt from PLIC
            source = kv_plic_claim_interrupt();

           (source) {
         KV_PLIC_SRC_UART:
        uart_isr();
            ;
         KV_PLIC_SRC_SPI:
        spi_isr();
            ;
    // ... handle other sources ...
```

```
    }

    // Complete interrupt
    kv_plic_complete_interrupt(source);
}
```

```
              (    ) {
    kv_uart_init(      );  // Configure baud rate

          ( ) {
           (kv_uart_rx_ready()) {
                    c = kv_uart_getc();
            kv_uart_putc(c);  // Echo character
        }
    }
}
```

```
                  (         addr,         *buf,        len) {
    kv_spi_init(       , KV_SPI_MODE0);  // 1 MHz, Mode 0

    kv_spi_select( );  // Assert CS[0]

    // Send READ command + 24-bit address
    kv_spi_transfer(SPI_CMD_READ);
    kv_spi_transfer((addr >>   ) &     );
    kv_spi_transfer((addr >>  ) &     );
    kv_spi_transfer(addr &     );

    // Read data
        (       i =  ; i < len; i++) {
        buf[i] = kv_spi_transfer(    );
    }

    kv_spi_deselect( );  // De-assert CS[0]
```

```
}
```

```
                    (        mem_addr,         data) {
    kv_i2c_init(      );  // 100 kHz

    kv_i2c_start();
    kv_i2c_write(EEPROM_ADDR <<  );  // Address + W bit
    kv_i2c_write(mem_addr);           // Memory address
    kv_i2c_write(data);               // Data byte
    kv_i2c_stop();

    // Wait for write cycle to complete (typical 5ms)
        (             i =  ; i <       ; i++);
}
```

```
                (     *dst,            *src,        len) {
    // Configure channel 0 for 1D transfer
    kv_dma_channel_config( , (        )src, (        )dst, len,
                          KV_DMA_CTRL_MODE_1D |
                          KV_DMA_CTRL_SRC_INC |
                          KV_DMA_CTRL_DST_INC);

    // Enable interrupt
    kv_dma_channel_enable_interrupt( );

    // Start transfer
    kv_dma_channel_start( );

    // Wait for completion (or use interrupt)
          (kv_dma_channel_busy( ));

    // Check for errors
       (kv_dma_channel_error( )) {
        // Handle error
    }
}
```

ICACHE_EN=1 !

DCACHE_EN=1

DCACHE_WRITE_BACK=1
DCACHE_WRITE_BACK=0

IB_DEPTH SB_DEPTH

cbo.flush
FENCE.I

FAST_MUL=1 FAST_DIV=1

```
debug_printf("Checkpoint: pc=0x%08x, x10=%d\n", read_csr(mepc), x10);
```

```
make WAVE=1 rtl-simple
gtkwave build/tb_kv32_soc.vcd
```

kv32_core.pc_if state_id alu_result_ex

imem_req_valid dmem_resp_valid axi_xbar

timer_irq external_irq trap_entry

```
make compare-simple
diff sim_trace.txt rtl_trace.txt
```

```
<cycle> <pc> <instruction> <disassembly>
```


			{VERSION[31:16], TX_DEPTH[15:8], RX_DEPTH[7:0]}

			{VERSION[31:16], TX_DEPTH[15:8], RX_DEPTH[7:0]}

			{VERSION[31:16], TX_DEPTH[15:8], RX_DEPTH[7:0]}

			{VERSION[31:16], NUM_CHANNELS[15:8], MAX_BURST_LEN[7:0]}

b

x

			0x0001_0120

n

x

			0x0001_0420

Table 53. Machine-Mode Interrupt Vectors

$

		LR/SC
		FENCE.I

mstatus

mie

mtvec

mscratch

mepc

mcause

mtval

mip

mtvec MRET

mcycle mcycleh

minstret minstreth

cycle cycleh

instret instreth

time timeh

A

LR.W SC.W

C

FENCE.I

FENCE.I

pmacfg0 0x7C0

pmacfg1 0x7C1

pmaaddr0 pmaaddr7 0x7C4 0x7CB

OFF TOR NA4 NAPOT

X C B

L

addr[31] = 1 "

addr[31] = 0 "

S

U

satp

medeleg mideleg

mtime mtimecmp msip

RV32IMAC + machine-mode traps + standard machine CSRs + atomic + compressed

FENCE.I

			axi_gpio.sv axi_timer.sv "
			pmacfg0/1 pmaaddr0 pmaaddr7